

Session 10: Context and Pipeline Parallelism

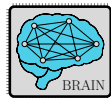
Splitting Sequence Length and Model Depth

Jonathan Lys

IMT Atlantique
September 19, 2026



IMT Atlantique
Bretagne - Pays de la Loire
École Mines-Télécom



Training Language Models: From First Principles to Efficient Serving

Lecture map

- 1 Context parallelism
- 2 Pipeline parallelism
- 3 Composing parallel dimensions

Guiding question

When width sharding is not enough, how do we divide long contexts and deep models without losing exactness or utilization?

What should be clear by the end?

- 1 Distinguish context parallelism from sequence parallelism.
- 2 Explain exact ring attention through online softmax.
- 3 Compute pipeline bubbles for a micro-batch schedule.
- 4 Map parallel dimensions to a real cluster topology.

Outline

- 1 Context parallelism
- 2 Pipeline parallelism
- 3 Composing parallel dimensions

Two techniques shard the sequence axis for different reasons

Sequence parallelism

Complements tensor parallelism by sharding token activations around norms, dropout, and elementwise operations.

Context parallelism

Splits a long context across devices so each rank stores only part of the queries and token activations.

The hard part

Attention couples every local query with keys and values that may live on every other rank.

Most layers are local after token sharding

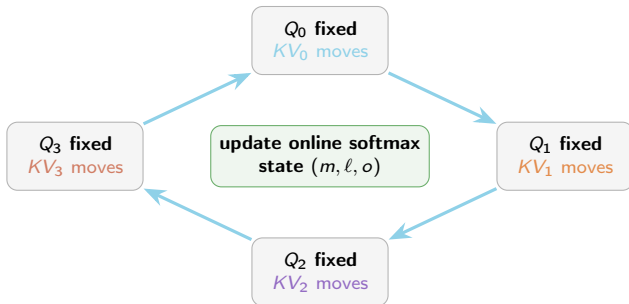
With T split over C context ranks, each rank owns roughly T/C positions.

- tokenwise normalization and MLPs operate locally
- residual additions operate locally
- attention needs remote key-value blocks
- some losses and metrics need global reductions

Memory effect

Token activation memory falls roughly with $1/C$, while communication appears inside every attention layer.

Ring attention rotates key-value blocks



after C rounds: every Q_r has seen all KV blocks

Memory behavior

Only a few KV blocks are resident per rank; no rank gathers the full context or materializes a global $T \times T$ score matrix.

Source: Hugging Face Ultra-Scale Playbook and Ring Attention

Online softmax makes blockwise attention exact

For one query row, maintain running maximum m , normalizer ℓ , and numerator o . For a new score block with maximum m_b :

$$m' = \max(m, m_b)$$

$$\ell' = e^{m-m'} \ell + \sum_j e^{s_j-m'}$$

$$o' = e^{m-m'} o + \sum_j e^{s_j-m'} v_j$$

Final attention is o'/ℓ' . Rescaling preserves contributions computed under earlier maxima.

Causal attention can imbalance ring work

Early query positions have few valid keys. Late queries have many. A naive contiguous partition gives ranks unequal useful work after masking.

Symptom

Some ranks wait while others process large valid attention regions.

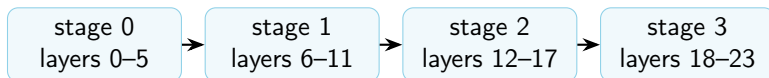
Mitigation

Interleave or pair early and late context blocks so valid attention work is balanced.

Outline

- ① Context parallelism
- ② Pipeline parallelism
- ③ Composing parallel dimensions

Pipeline parallelism assigns layer ranges to stages



Why micro-batches?

Split a global batch into M micro-batches so different stages can work on different micro-batches concurrently.

Pipeline bubbles shrink as micro-batches increase

For P balanced stages and M forward-only micro-batches:

$$\text{ideal utilization} \approx \frac{M}{M + P - 1}$$

Worked example

$P = 4$, $M = 8$ gives $8/(8 + 3) \approx 73\%$ idealized forward utilization. With $M = 32$, it rises to about 91%.

Trade-off

More micro-batches reduce bubbles but can create smaller, less efficient kernels and more activation bookkeeping.

AFAB and 1F1B trade memory for schedule complexity

All-forward-all-backward

- simple schedule
- many outstanding activations
- higher peak memory

One-forward-one-backward

- after warmup, alternate directions
- fewer stored activations
- more coordination

Both still have warmup and cooldown bubbles. Interleaving virtual stages can reduce them further at added complexity.

The slowest stage sets pipeline throughput

$$t_{\text{cycle}} \approx \max_s t_s$$

- Equal layer counts need not mean equal work.
- Embeddings, output projection, and activation sizes can skew stages.
- Stage boundaries determine activation communication volume.
- Profile stage time and memory before moving boundaries.

Checkpoint: schedule arithmetic

Think, then compare

For six stages and twelve micro-batches, estimate idealized forward utilization.
Name two real effects that make measured utilization lower.

Outline

- ① Context parallelism
- ② Pipeline parallelism
- ③ Composing parallel dimensions

The device count factorizes across parallel groups

$$N_{\text{GPU}} = D \times T_p \times C \times P$$

Dimension	Splits	Communication pattern
data / FSDP	batch and model state	gradient or shard collectives
tensor	hidden width and heads	frequent activation collectives
context	sequence positions	key-value exchange per layer
pipeline	layer depth	point-to-point activations

Map the chattiest group to the fastest fabric

- 1 Use tensor parallelism inside a high-bandwidth GPU island.
- 2 Add context parallelism when token activations or attention exceed local memory.
- 3 Use pipeline parallelism when model depth spans nodes.
- 4 Replicate the resulting model group with data parallelism when batch and devices remain.

Selection rule

Choose the smallest degree on each axis that resolves a concrete memory or compute constraint.

Four cases lead to different strategies

Binding constraint	First strategy to evaluate
model fits, short context, more GPUs	DDP or FSDP replicas
one wide layer does not fit	tensor parallelism
weights fit, 128k-token examples do not	context parallelism plus checkpointing
model exceeds one node	pipeline across nodes, TP within node

Checkpoint: cluster design

Think, then compare

Design a layout for 32 GPUs arranged as four 8-GPU nodes. State the memory constraint, proposed D , T_p , C , P , and which communication crosses node boundaries.

The durable ideas

- 1 Context parallelism shards tokens and uses exact blockwise attention with online softmax.
- 2 Pipeline parallelism shards depth, while micro-batches trade bubble size against memory and kernel efficiency.
- 3 Load balance matters across causal context blocks and pipeline stages.
- 4 Parallel dimensions should follow the workload constraint and physical network hierarchy.